

KMIP on PKCS#11

Enterprise KMS Gap Matrix

Market requirements, coverage, and the route for every gap

Document	Requirements assessment against the commercial KMS market
Assessed at	2964dee — 816 tests green
Requirements from	Thales CipherTrust Manager, Entrust KeyControl, Fortanix DSM, Utimaco ESKM, Bloombase KeyCastle, Cosmian/Eviden KMS, Securosys CyberVault KMS, HashiCorp Vault, AWS/Azure/GCP KMS; NIST SP 800-57 and SP 800-152
Features assessed	82 across 7 domains
Result	32 covered · 18 partial · 32 not covered · 23 need work outside KMIP
Date	12 September 2026

1 How to read this document

This is a buy-or-build assessment, not a feature advertisement. Each row names something a high-end key management platform is expected to do, says whether this project does it, and — where it does not — names the route that would actually close the gap.

The four verdicts

Verdict	Means
Covered	The capability exists and was demonstrated against a live token, with regression tests in the suite. It does not mean it matches a commercial product feature-for-feature at enterprise scale.
Partial	Something real is there, but an enterprise buyer would find a named limit. The route column says what the limit is.
Not covered	Absent. The route column says what building it would involve.
Outside KMIP	Not a verdict but a marker on the route: this gap cannot be closed by writing more KMIP, because the specification defines nothing for it.

That last marker is the most useful thing on the page. KMIP standardises key objects and the operations on them — it does not standardise the service around them. Identity federation, tenancy, quorum workflow, reporting, cloud key import and SIEM export have no wire representation at all, which is why every commercial product bolts a REST administration plane onto its KMIP server. A KMIP layer is an interoperability surface, not a product.

Where a row says a capability belongs to the token rather than to this code — FIPS validation, certified entropy — that is a procurement decision, not an engineering backlog item, and is marked as such rather than quietly counted as a gap this project could close.

2 Summary

82 features were assessed. 32 are covered, 18 partially, and 32 are not covered at all. Of everything short of full coverage, 23 cannot be reached through the KMIP protocol.

Domain	Features	Covered	Partial	Not covered
Key lifecycle and cryptography	15	8	4	3
Protocol and ecosystem integration	15	1	5	9
Identity and access control	13	6	1	6
Audit, compliance and assurance	11	3	2	6
Availability, scale and recovery	11	3	3	5
Operations and observability	9	5	1	3
Platform hardening	8	6	2	0
TOTAL	82	32	18	32

Where the project is strong

The cryptographic core and the platform hardening are close to complete, and several parts are done properly rather than nominally: the restore procedure proves a stored secret actually decrypts rather than assuming it, dual control is enforced in the store so the requester cannot self-approve however the approval arrives, and the audit chain names the exact row when verification fails.

Where it is weak

Everything that makes a key server into a product an enterprise buys: no REST API, no console, no directory integration, no tenancy, no clustering, and no validated hardware. None of these are cryptographic shortcomings. They are the service around the cryptography, and that is a different body of work.

3 Key lifecycle and cryptography

15 features — 8 covered, 4 partial, 3 not covered.

Feature	What the market expects	Status	Route, if not covered here
HSM-backed storage, keys never leave	Key material generated and held in a certified token; the KMS holds references, not secrets.	Covered	Every key is a PKCS#11 object; only pkcs11_shim/ imports the binding.
Full lifecycle state machine	Pre-Active → Active → Deactivated / Compromised → Destroyed, with state-gated usage.	Covered	Enforced per operation. Decrypt stays legal on a Deactivated key so retiring one does not strand data.
Crypto-period enforcement	Keys deactivate on policy, not when someone remembers (NIST SP 800-57).	Covered	Background scheduler; every action audited as system:scheduler.
Automatic rotation	Expiring keys replaced automatically, with lineage preserved.	Partial	Symmetric keys only. Asymmetric rotation would reuse the existing ReKeyKeyPair handler — in-project, no protocol change.
Key lineage and versioning	Replacement keys linked to predecessors so data can be traced to the key that protected it.	Covered	Link_ReplacementKey / Link_ReplacedKey, identical for client-driven and scheduled rotation.
Algorithm breadth	AES, RSA, EC, and the curve and hash families an enterprise estate actually uses.	Partial	15 of 40 KMIP algorithms work on this token. The shim probes and gates; a richer token activates more with no code change.
Post-quantum (ML-KEM, ML-DSA, SLH-DSA)	FIPS 203/204/205 support, now the leading procurement question.	Not covered	Needs both halves: a PQC token (SoftHSM's ML-DSA/ML-KEM work is in the development tree, not in any release) and KMIP 3.0, whose CSD02 adds the algorithms plus Encapsulate/Decapsulate. Vendor-range enums would work but interoperate with nothing.
Hardware RNG	Entropy from certified hardware, exposed to clients.	Covered	RNGRetrieve and RNGSeed draw from the token. Certification is the token's property, not this code's.
Key derivation	Derive keys from existing material without exposing either.	Covered	DeriveKey, performed inside the token.
Key wrapping / secure import-export	Move keys between systems without ever seeing plaintext.	Covered	AES key wrap, and a wrapped Get via KeyWrappingSpecification.
Split knowledge / M-of-N shares	Key material split so no one custodian can reconstruct it.	Partial	XOR only, and strictly N-of-N. Shamir threshold sharing would slot into the existing SplitKeyMethod enum — inside KMIP.

Formal key ceremony	Scripted, witnessed, attested generation for root keys.	Not covered	Outside KMIP — A ceremony runbook plus signed kmip-admin transcripts. The cryptographic primitive already exists as dual control; what is missing is the procedure and its evidence.
Bulk and batch operations	Many operations per round trip, for provisioning at scale.	Partial	Batching works, atomicity does not — a mid-batch failure leaves earlier items applied. KMIP defines no rollback, so all-or-nothing would be a local transaction around the dispatcher.
Verifiable destruction	Zeroization on the token, evidenced.	Covered	Destroy zeroizes the PKCS#11 object; the audit row survives to resolve the identifier.
Key-bound usage policy enforced in hardware	The approval quorum, time-lock or usage limit travels with the key inside the token, so the policy survives compromise of the management server.	Not covered	Outside KMIP — Dual control here is enforced by the server: an attacker who owns the server owns the policy. Moving it into the key needs a token whose attributes carry approval rules and time-locks — Securosys Smart Key Attributes are the reference implementation — and SoftHSM has no equivalent. Token capability, not code.

4 Protocol and ecosystem integration

15 features — 1 covered, 5 partial, 9 not covered.

Feature	What the market expects	Status	Route, if not covered here
KMIP server	The interoperability baseline every enterprise KMS ships.	Covered	41 of 53 KMIP 2.1 operations; the other 12 are session, async and vendor operations.
KMIP interoperability certification	Demonstrated interop against other vendors' clients and servers.	Not covered	OASIS conformance test cases are mapped in the suite, but the project has never been through an interop event. This is testing and process, not new code.
KMIP 3.0	The current specification line, and where PQC lives.	Not covered	Enum and operation additions on top of 2.1. Substantial but entirely in-protocol.
PKCS#11 provider for applications	Applications plugging straight in through the interfaces they already speak — PKCS#11, and alongside it JCE and Microsoft CNG.	Not covered	Outside KMIP — This project consumes PKCS#11; it does not expose it. The route is a thin PKCS#11 library that fronts the KMIP server, which is how commercial products serve both interfaces.
REST / JSON API	Developer-facing API alongside KMIP; universal in the market.	Not covered	Outside KMIP — KMIP defines no REST binding. An HTTP facade over the existing dispatcher would reuse identity, RBAC and audit unchanged. This is the highest-leverage single gap on the page.
Cloud BYOK (AWS, Azure, GCP)	Generate on-premise, import into a cloud KMS, keep custody of the source.	Not covered	Outside KMIP — Per-cloud connectors calling each provider's import API. Nothing in KMIP describes this.
Cloud EKM / XKS / hold-your-own-key	Cloud services calling back to your KMS for every operation.	Not covered	Outside KMIP — Implement the AWS XKS proxy API or the GCP EKM API as HTTP services in front of the same store.
Database TDE integration	Oracle, SQL Server, MongoDB, Db2 taking keys from the KMS.	Partial	These speak KMIP, so the path exists — but none has been tested against this server. The work is profile conformance and vendor validation, not new protocol.
Storage, backup and VM integrations	VMware, NetApp, Veeam, tape libraries.	Partial	Same position as database TDE: KMIP-capable, unvalidated.
Secrets management	Arbitrary secrets with versioning, leases and dynamic issuance.	Partial	Outside KMIP — SecretData objects and ObtainLease cover static custody. A Vault-style secrets engine —

			dynamic credentials, templating — is a different product surface.
Tokenization / format-preserving encryption	FPE for PAN and PII, common in payments deployments.	Not covered	Outside KMIP — Neither a KMIP operation nor typically a token mechanism. A separate service implementing FF1/FF3 with keys held here.
Certificate lifecycle	CA issuance, CSR handling, renewal, revocation checking.	Partial	Certify, ReCertify and Validate exist but issue self-signed certificates only, with no CRL or OSCP. A real PKI means fronting a CA such as EJBCA; KMIP's Certify was never a CA.
Encryption as a service (data-plane API)	Applications send data rather than fetching keys: encrypt, decrypt, sign and HMAC, plus data-key issuance and re-wrap to a newer key version.	Partial	Encrypt, Decrypt, Sign, MAC and Hash already execute inside the token, so the capability exists over KMIP. Missing are a data-key call returning the wrapped and plaintext pair together, re-wrap to a newer version, and any of it over HTTP.
KMIP JSON and XML encodings	The same protocol in the JSON and XML encodings OASIS defines, carried over HTTPS, so a client can speak KMIP without a TTLV codec.	Not covered	In-protocol and well specified: a second codec beside core/ttlv.py plus an HTTP binding, reusing the object model unchanged. Worth doing when a client asks — nothing depends on it today.
OpenAPI specification and generated clients	A machine-readable description of the API with an interactive explorer, and client libraries generated from it rather than hand-written.	Not covered	Outside KMIP — There is no API to describe yet. Generating both the document and the clients from the router's route table is what stops either drifting from the server.

5 Identity and access control

13 features — 6 covered, 1 partial, 6 not covered.

Feature	What the market expects	Status	Route, if not covered here
Per-identity authentication	Individually revocable credentials; no shared secret.	Covered	Salted script credentials, verified per request, with a short-lived cache for successes only.
Enterprise IdP — LDAP/AD, SAML, OIDC	Users and services authenticating against the corporate directory.	Not covered	Outside KMIP — KMIP's Credential structure has no federation concept. Map at the edge: an OIDC-validating front door, or directory-backed identities in the store. The mTLS path already proves the pattern — an external assertion resolved to a provisioned identity.
mTLS certificate identity	Client certificates as principals.	Covered	A verified CN is accepted only when it names a provisioned identity — a CA-signed certificate cannot invent a principal.
API keys / service accounts	Long-lived machine credentials distinct from human logins.	Not covered	A new Credential type, or token identities in the store. Partly expressible in KMIP, mostly local.
Role-based access control	Operations restricted by role.	Covered	Roles with opt-in per-operation allowlists.
Groups	Access following team membership rather than per-person grants.	Covered	Grants may name group:<name>; leaving the group withdraws access.
Per-object access grants	Delegation of one key to one principal.	Covered	read / full permission levels per object.
Attribute or policy-based access	Conditions on time, network, purpose; a policy engine.	Not covered	Outside KMIP — A policy evaluation hook in the dispatcher, before the handler — the same insertion point dual control already uses.
Dual control / quorum approval	Destructive or disclosing operations need M-of-N sign-off.	Covered	Destroy and Export refuse on first attempt; approval is out of band, the requester can never self-approve, and an approval is spent by one attempt.
Separation of duties	Key administrators cannot use the keys they administer.	Partial	The admin role currently bypasses every check. Splitting it into security-admin and key-admin is local work with no protocol impact.
Multi-tenancy and namespaces	Unrelated tenants sharing a deployment without seeing each other.	Not covered	Outside KMIP — Object names, Locate results, audit and quotas are all global. A tenant boundary reaches into

			every query, the audit log and the CLI — a phase of work, not a flag. Isolation today means one deployment per tenant.
Per-tenant quotas	Caps on objects and operations.	Not covered	Outside KMIP — Follows tenancy; meaningless before it.
Connection and rate limiting	Throttling and connection caps as DoS protection.	Not covered	Outside KMIP — The listener takes a backlog of 16 with no throttle. A bounded connection semaphore plus per-identity rate limiting, or a reverse proxy in front.

6 Audit, compliance and assurance

11 features — 3 covered, 2 partial, 6 not covered.

Feature	What the market expects	Status	Route, if not covered here
Tamper-evident audit log	Every operation attributable, with tampering detectable.	Covered	Hash-chained and trigger-protected; the verifier names the exact broken row. Reads are recorded too.
External audit anchoring	Evidence that survives an attacker with file access.	Not covered	Outside KMIP — Ship entries to an append-only collector, or notarize periodic digests. A consistent rewrite of the whole local log currently leaves no trace.
SIEM integration	Syslog/CEF to Splunk, Sentinel, QRadar.	Not covered	Outside KMIP — A syslog handler, or a shipper reading kmip-admin audit list --json. Small work, and it is what auditors ask for first.
Retention and archival	Aged entries archived, not silently dropped.	Covered	Pruning returns what it removes and refuses to run on a log that already fails verification.
FIPS 140-2/3 validated HSM	Validated hardware, frequently a procurement gate.	Not covered	Not an engineering task. SoftHSM2 is not validated; the PKCS#11 boundary makes swapping in a validated token cheap, but the validation belongs to the token.
Common Criteria / eIDAS	Certification for regulated European deployments.	Not covered	As above — a property of the chosen token and of a certification programme.
Compliance reporting	PCI DSS, GDPR, HIPAA evidence packs.	Not covered	Outside KMIP — A reporting layer over the audit log and key inventory. Depends on the REST API to be useful.
Key inventory and discovery	What keys exist, who owns them, what state they are in.	Partial	Locate answers this over the wire and kmip-admin can list, but there is no inventory report or export.
Cryptoperiod policy alignment	Demonstrable NIST SP 800-57 lifecycle handling.	Covered	Deactivation Date as the cryptoperiod, enforced and audited.
Crypto-agility reporting	Which algorithms are in use and where, for migration planning.	Partial	The capability probe knows what the token supports and every object records its algorithm; nothing surfaces it as a report.
Certificate discovery and expiry monitoring	Find the certificates in use across the estate, track expiry, and warn before something goes	Not covered	Outside KMIP — This server knows only the certificates registered with it. Discovery means

	down.		scanning endpoints and stores it does not own — a separate scanner feeding the inventory, not a KMIP operation.
--	-------	--	---

7 Availability, scale and recovery

11 features — 3 covered, 3 partial, 5 not covered.

Feature	What the market expects	Status	Route, if not covered here
HA clustering	Active/active nodes behind a VIP, no single point of failure.	Not covered	Outside KMIP — The blocker is storage, not protocol: SQLite has no multi-writer story. Postgres plus leader election for the scheduler.
Multi-site replication and DR	Geographic redundancy with a defined RPO.	Not covered	Outside KMIP — Follows the storage backend.
HSM failover and pooling	Survive the loss of one token; spread load across several.	Not covered	A pool at the shim layer. Deliberately unbuilt — verifying it needs a second token, and an untested failover path invites reliance.
Horizontal throughput	Scale with cores and nodes.	Partial	Pre-fork workers give about 1.5×, bounded by the serial audit chain — the measured cost of tamper-evidence. Per-shard chains or an external append-only service would lift it.
Enterprise database backend	Postgres or Oracle rather than an embedded file.	Not covered	The store is heavily SQLite-specific (PRAGMA user_version, json_extract, RAISE(ABORT) triggers), so this is a persistence-layer rewrite.
Scheduled and offsite backup	Automatic snapshots shipped off the box.	Partial	Backup and restore are fully tooled and verified; scheduling and shipping are left to cron and the operator.
Restore that proves itself	A restore verified, not assumed.	Covered	Checks the master key is present, the audit chain verifies, and a stored secret actually decrypts. Token pairing is enforced.
Zero-downtime certificate rotation	Renew TLS material without dropping connections.	Covered	SIGHUP reload; systemctl reload costs nothing.
Rolling upgrades	Version-to-version upgrade without data loss.	Covered	Versioned migrations gated on PRAGMA user_version, each committing with its bump so an interrupted upgrade resumes. Single-node only.
Kubernetes / container deployment	A supported image and manifests.	Partial	A two-stage Dockerfile exists and its inputs are checked, but it has never been executed — this environment blocks Docker Hub. No Helm chart or operator.
Confidential	Run inside a TEE or	Not covered	Outside KMIP —

computing deployment	confidential VM with remote attestation, so the host operator cannot read process memory.		Mostly deployment and attestation rather than code, but not free: it needs a store designed for an untrusted host, and an attestation a client can check before it trusts the service.
---------------------------------	---	--	--

8 Operations and observability

9 features — 5 covered, 1 partial, 3 not covered.

Feature	What the market expects	Status	Route, if not covered here
Health and readiness probes	Orchestrator-grade liveness and readiness.	Covered	Readiness gates on the listener as well as the HSM and database, so traffic is not routed at a port that is not open.
Metrics	Prometheus-compatible operational telemetry.	Covered	Operation counters, latency totals and auth failures. Latency is a running total rather than histogram buckets, and only worker 0 serves the endpoint, so quantiles and fleet totals need the collector.
Structured logging	Machine-parseable logs that survive aggregation.	Covered	JSON with tracebacks in a field rather than trailing newlines.
Configuration-driven deployment	A config file and a service unit, not a program to maintain.	Covered	One YAML file, strict validation naming the offending key, and --check to validate without serving.
Administrative CLI	Scriptable administration.	Covered	kmip-admin covers identities, roles, groups, grants, permissions, approvals, cryptoperiods, audit, backup and master-key rotation.
Web console	A GUI for operators and auditors — universal in commercial products.	Not covered	Outside KMIP — Needs the REST admin API first; the console is then a client of it.
Self-service developer portal	Teams provisioning their own keys within policy.	Not covered	Outside KMIP — Follows the console and tenancy.
Alerting	Expiry and anomaly alerts pushed to on-call.	Partial	Approaching expiry is logged and audited, and counters are exported, but nothing pushes. Alertmanager rules over the existing metrics would close most of it.
Distributed tracing	Spans across the service and into the token, usually OpenTelemetry, so a slow request can be explained rather than guessed at.	Not covered	Structured logs carry a request identifier, which answers what happened but not where the time went. Nothing exports spans.

9 Platform hardening

8 features — 6 covered, 2 partial, 0 not covered.

Feature	What the market expects	Status	Route, if not covered here
TLS enforced by default	No accidental plaintext.	Covered	Refuses to start in the clear without an explicit opt-out that warns and names the address. TLS 1.2 floor.
Metadata encrypted at rest	A stolen database yields nothing.	Covered	AES-256-GCM envelopes under a non-extractable HSM master key, with a post-upgrade scrub that reaches the WAL sidecar.
Master key rotation	Re-key the protection layer without downtime or data loss.	Covered	Row-by-row re-encryption; a partly rotated store stays readable and the old key is destroyed only after the last row moves.
Non-extractable key enforcement	Keys that cannot be read out, enforced by hardware.	Covered	Set on the token, so it holds regardless of what this server believes.
Request bounds and DoS resistance	Hostile input cannot exhaust the server.	Partial	Request size is capped before the body is read, the TLS handshake is off the accept loop under a timeout, and TTLV nesting is bounded — but there is still no connection cap or rate limit.
Secrets kept out of configuration	Credentials from a secrets manager, not a file in git.	Covered	PIN from file or environment; inline works, warns at every start, and is redacted from any config dump.
Hardened service	Least privilege at the OS level.	Covered	systemd unit with no capabilities, strict filesystem protection, syscall filtering and a dedicated user.
Sealed startup with quorum unseal	The service starts sealed and its state stays unreadable until unsealed — by a quorum of operator-held shares, or automatically by a trusted device — and the shares can be re-issued without downtime.	Partial	The auto-unseal half exists: the store is encrypted under a non-extractable HSM key, so a stolen database is inert without the token. There is no operator quorum and no re-issue ceremony — whoever holds the token PIN unseals it alone.

10 What to build first

Ordered by how much each unlocks, not by how visible it is.

A REST administration API

The highest-leverage single item on the page. A web console, a self-service portal, compliance reporting, SIEM-friendly export and most integrations are all clients of an API that does not exist yet — five separate gaps collapse into one piece of work. It would reuse the existing identity, RBAC and audit layers unchanged, because those already sit below the protocol rather than inside it.

SIEM export

Small work, and the thing auditors ask for first. A syslog handler, or a shipper reading the existing JSON audit output. It does not need the REST API to be useful.

Connection and rate limiting

The listener still takes a backlog of sixteen with no throttle, which is a denial-of-service exposure rather than a missing feature. A bounded connection semaphore and per-identity rate limiting, or a reverse proxy in front of it.

A PostgreSQL storage backend

The blocker beneath clustering, replication and disaster recovery — none of which are protocol problems. SQLite has no multi-writer story, and the store is heavily SQLite-specific, so this is a persistence-layer rewrite rather than a driver swap.

Multi-tenancy

A phase of work, not a flag: object names, Locate results, the audit log, quotas and the administrative CLI all assume a single tenant. Worth doing only once there is a second tenant to serve; until then, one deployment per tenant is a legitimate answer.

Post-quantum readiness

Needs both halves and neither is here yet. A token that implements ML-KEM and ML-DSA — SoftHSM2's work is in its development tree, not in any release — and KMIP 3.0, whose draft adds the algorithms plus Encapsulate and Decapsulate. Vendor-range enumerations would work locally and interoperate with nothing, which is worse than waiting.

Two items are deliberately absent from this list. FIPS 140-3 and Common Criteria validation belong to the token, so the work is procurement and testing rather than development. And HSM failover is unbuilt for a reason already recorded elsewhere in this project's documentation: verifying it needs a second token, and an untested failover path invites exactly the reliance it cannot support.

11 Sources

The requirement list was assembled from published vendor capability documentation and the two NIST publications that define what a key management system owes its operator. The coverage column comes entirely from this repository.

Products whose documentation was read

Product	What it contributed
Thales CipherTrust Manager	HA clustering and intelligent key sharing, multi-tenancy, REST API, key lifecycle scope, deployment options.
Fortanix Data Security Manager	Quorum approval policy, BYOK and BYOKMS, cloud EKM and AWS XKS, tokenization, FIPS 140-2 Level 3 appliance.
Entrust KeyControl	Distributed isolated vaults for data residency; a compliance dashboard covering PCI DSS, HIPAA and NIST SP 800-130; automated rotation, backup and revocation; FIPS 140-3 and Common Criteria EAL4+ HSM integration.
Utimaco Enterprise Secure Key Manager	Enterprise deployment expectations, HA and DR framing.
AWS KMS, Azure Key Vault Managed HSM, Google Cloud KMS	External key store patterns — AWS XKS, Azure external key management, Google Cloud EKM — BYOK import, rotation intervals, and the FIPS levels each is validated to.
Bloombase KeyCastle	The appliance-era operations surface: web console, SNMP v1/v2c/v3 and syslog with rotation and archival, multiple certificate authorities, X.509 and PKCS#12 import and export, and a published interoperability programme — which is what KMIP interoperability certification looks like in practice.
Cosmian / Eviden KMS	KMIP in JSON as well as binary TTLV; post-quantum algorithms shipping now; an encryption oracle rather than key delivery; SQLite, PostgreSQL, MySQL, MariaDB and Redis backends; OpenTelemetry; OpenAPI 3.1 with an interactive explorer; and deployment inside a confidential VM.
Securosys CyberVault KMS	Smart Key Attributes — quorum approval, time-locks and timeouts bound to the key and enforced inside the HSM rather than by the management server; certificate discovery and expiry monitoring; posture checks for unused keys and weak configurations; ML-KEM, ML-DSA, SLH-DSA and LMS; REST, KMIP, PKCS#11, JCE and CNG side by side.
HashiCorp Vault	The vault architecture: a service that starts sealed and unseals on a quorum of Shamir shares or a trusted device; namespaces with their own policies; identity entities and groups; rate-limit quotas; performance and disaster-recovery replication; encryption as a service through the transit engine; and KMIP as one more front end.

Standards and specifications

Source	What it contributed
OASIS KMIP 2.1, and the 3.0 draft (CSD02, May 2026)	Protocol scope, and what the protocol deliberately leaves out. The 3.0 draft is where ML-KEM, ML-DSA, SLH-DSA and the Encapsulate / Decapsulate operations arrive.

NIST SP 800-57 Part 1	Key lifecycle states and cryptoperiod expectations.
NIST SP 800-152 (CKMS profile)	Backup, recovery, audit and disaster-recovery requirements.
This repository at 2964dee	Every entry in the coverage and route columns.

A note on method, since it bears on how much weight the requirement list carries. The nine products above were chosen because they span the category: software-first platforms, appliance vendors, an HSM-centric control plane, a source-available implementation, a vault architecture, and the three cloud services. Their published documentation describes what each vendor chooses to advertise, so the list is a fair picture of what the market expects a KMS to do and not a substitute for a hands-on evaluation of any one product. Where several vendors describe the same capability in different words, it appears here once.

The four products added in this revision were researched through indexed summaries of their documentation rather than the pages themselves: the network policy in the environment this document is built in blocks all four vendor documentation domains. That is a real limit on depth and it is recorded here rather than glossed over — it is also why the provenance document marks only what a source's documentation was actually seen to name.

One capability was read and deliberately not made a requirement. Securosys ships a conversational assistant that generates and manages keys from natural-language instructions. It is a real feature, but a requirement list is a statement about what a key management system owes its operator, and an assistant that can issue privileged operations is not yet something the category expects — or has agreed how to audit. It is noted here so the omission is visible as a judgement rather than an oversight.

This document is generated. Re-run `generate_kms_gap_matrix.py` after closing a gap, and update `ASSESSED_AT` at the top of that file so a reader can tell whether the coverage column still describes the code.